MLton

Elaborate

Elaborate is a translation pass from the AST IntermediateLanguage to the CoreML IntermediateLanguage.

Description

This pass performs type inference and type checking according to the Definition. It also defunctorizes the program, eliminating all module-level constructs.

Implementation

- elaborate.sig
- elaborate.fun
- elaborate-env.sig
- elaborate-env.fun
- elaborate-modules.sig
- elaborate-modules.fun
- elaborate-core.sig
- elaborate-core.fun
- elaborate

Details and Notes

At the modules level, the Elaborate pass:

- elaborates signatures with interfaces (see interface.sig and interface.fun)

  The main trick is to use disjoint sets to efficiently handle sharing of tycons and of structures and then to copy signatures as dags rather than as trees.

- checks functors at the point of definition, using functor summaries to speed up checking of functor applications.

  When a functor is first type checked, we keep track of the dummy argument structure and the dummy result structure, as well as all the tycons that were created while elaborating the body. Then, if we later need to type check an application of the functor (as opposed to defunctorize an application), we pair up tycons in the dummy argument structure with the actual argument structure and then replace the dummy tycons with the actual tycons in the dummy result structure, yielding the actual result structure. We also generate new tycons for all the tycons that we created while originally elaborating the body.

- handles opaque signature constraints.

  This is implemented by building a dummy structure realized from the signature, just as we would for a functor argument when type checking a functor. The dummy structure contains exactly the type information that is in the signature, which is what opacity requires. We then replace the variables (and constructors) in the dummy structure with the corresponding variables (and constructors) from the actual structure so that the translation to CoreML uses the right stuff. For each tycon in the dummy structure, we keep track of the corresponding type structure in the actual structure. This is used when producing the CoreML types (see `expandOpaque` in `type-env.sig` and `type-env.fun`).

  Then, within each `structure` or `functor` body, for each declaration (`<dec>` in the Standard ML grammar), the Elaborate pass does three steps:

  1. ScopeInference
  2.
     - PrecedenceParse
     - `_{ex,im}port` expansion
     - profiling insertion
     - unification
  3. Overloaded {constant, function, record pattern} resolution

Defunctorization

The Elaborate pass performs a number of duties historically assigned to the Defunctorize pass.

As part of the Elaborate pass, all module level constructs (`open`, `signature`, `structure`, `functor`, long identifiers) are removed. This works because the Elaborate pass assigns a unique name to every type and variable in the program. This also allows the Elaborate pass to eliminate `local` declarations, which are purely for namespace management.

Examples

Here are a number of examples of elaboration.

- All variables bound in `val` declarations are renamed.

```
val x = 13
val y = x

val x_0 = 13
val y_0 = x_0
```

- All variables in `fun` declarations are renamed.

```
fun f x = g x
and g y = f y

fun f_0 x_0 = g_0 x_0
and g_0 y_0 = f_0 y_0
```

- Type abbreviations are removed, and the abbreviation is expanded wherever it is used.

```
type 'a u = int * 'a
type 'b t = 'b u * real
fun f (x : bool t) = x

fun f_0 (x_0 : (int * bool) * real) = x_0
```

- Exception declarations create a new constructor and rename the type.

```
type t = int
exception E of t * real

exception E_0 of int * real
```

- The type and value constructors in datatype declarations are renamed.

```
datatype t = A of int | B of real * t

datatype t_0 = A_0 of int | B_0 of real * t_0
```

- Local declarations are moved to the top-level. The environment keeps track of the variables in scope.

```
val x = 13
local val x = 14
in val y = x
end
val z = x

val x_0 = 13
val x_1 = 14
val y_0 = x_1
val z_0 = x_0
```

- Structure declarations are eliminated, with all declarations moved to the top level. Long identifiers are renamed.

```
structure S =
   struct
      type t = int
      val x : t = 13
   end
val y : S.t = S.x

val x_0 : int = 13
val y_0 : int = x_0
```

- Open declarations are eliminated.

```
val x = 13
val y = 14
structure S =
   struct
     val x = 15
   end
open S
val z = x + y

val x_0 = 13
val y_0 = 14
val x_1 = 15
val z_0 = x_1 + y_0
```

- Functor declarations are eliminated, and the body of a functor is duplicated wherever the functor is applied.

```
functor F(val x : int) =
   struct
     val y = x
   end
structure F1 = F(val x = 13)
structure F2 = F(val x = 14)
val z = F1.y + F2.y

val x_0 = 13
val y_0 = x_0
val x_1 = 14
val y_1 = x_1
val z_0 = y_0 + y_1
```

- Signature constraints are eliminated. Note that signatures do affect how subsequent variables are renamed.

```
val y = 13
structure S : sig
                 val x : int
              end =
   struct
      val x = 14
      val y = x
   end
open S
val z = x + y

val y_0 = 13
val x_0 = 14
val y_1 = x_0
val z_0 = x_0 + y_0
```

Last updated Thu Oct 21 15:53:06 2021 -0400 by Matthew Fluet. Log Edit